

Data Layer Design

Data Layer Design

Strategies for managing persistence:

- Automatic generation of persistence
- Direct design of persistence
- References

Automatic generation of persistence

Strategies for managing persistence:

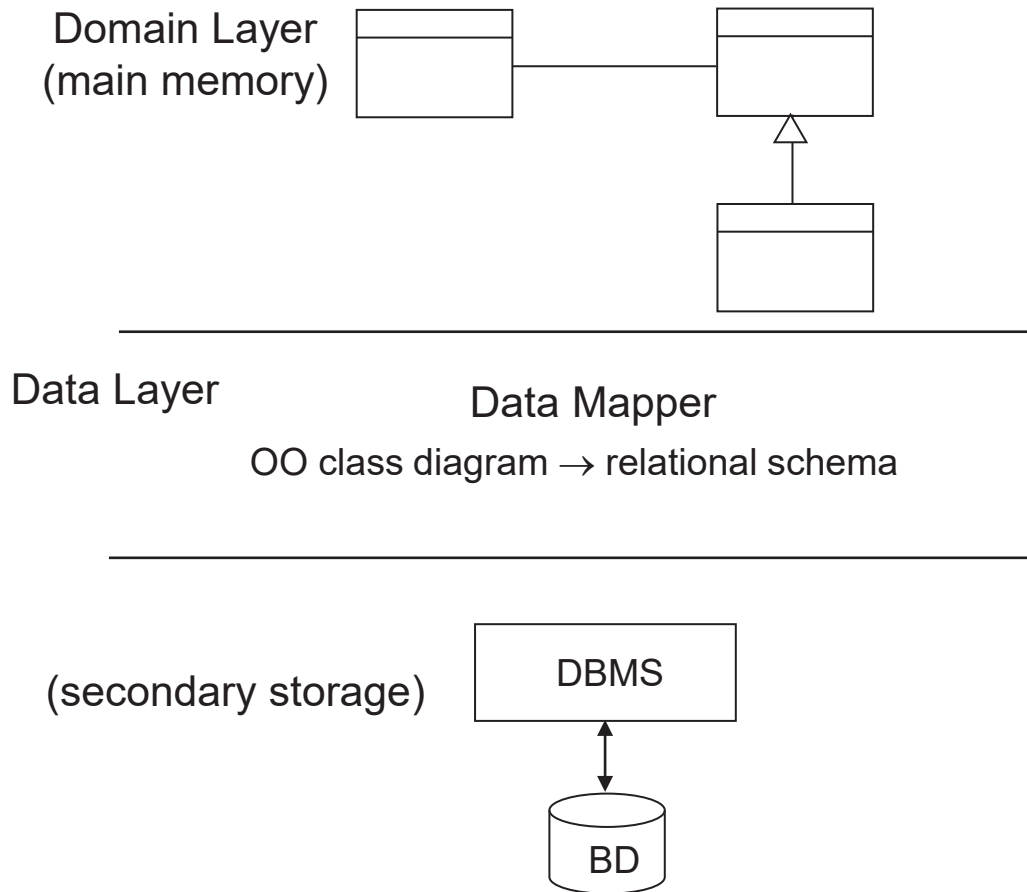
Automatic generation of persistence

- Automatic generation of persistence:
 - Software system that makes automatically the required translation for storing objects' data in secondary storage
 - Converts the updates made in main memory into updates in secondary storage
 - It requires the designer to specify the correspondence among the object-oriented class diagram and the data base schema
- Characteristics:
 - Persistence management is transparent to the designer
 - DBMS functionalities may not be fully exploited

It is used jointly with the Domain Model design pattern
(as it has been done in Unit 3.2)

Automatic generation of persistence:

Domain Model and Data Mapper design pattern



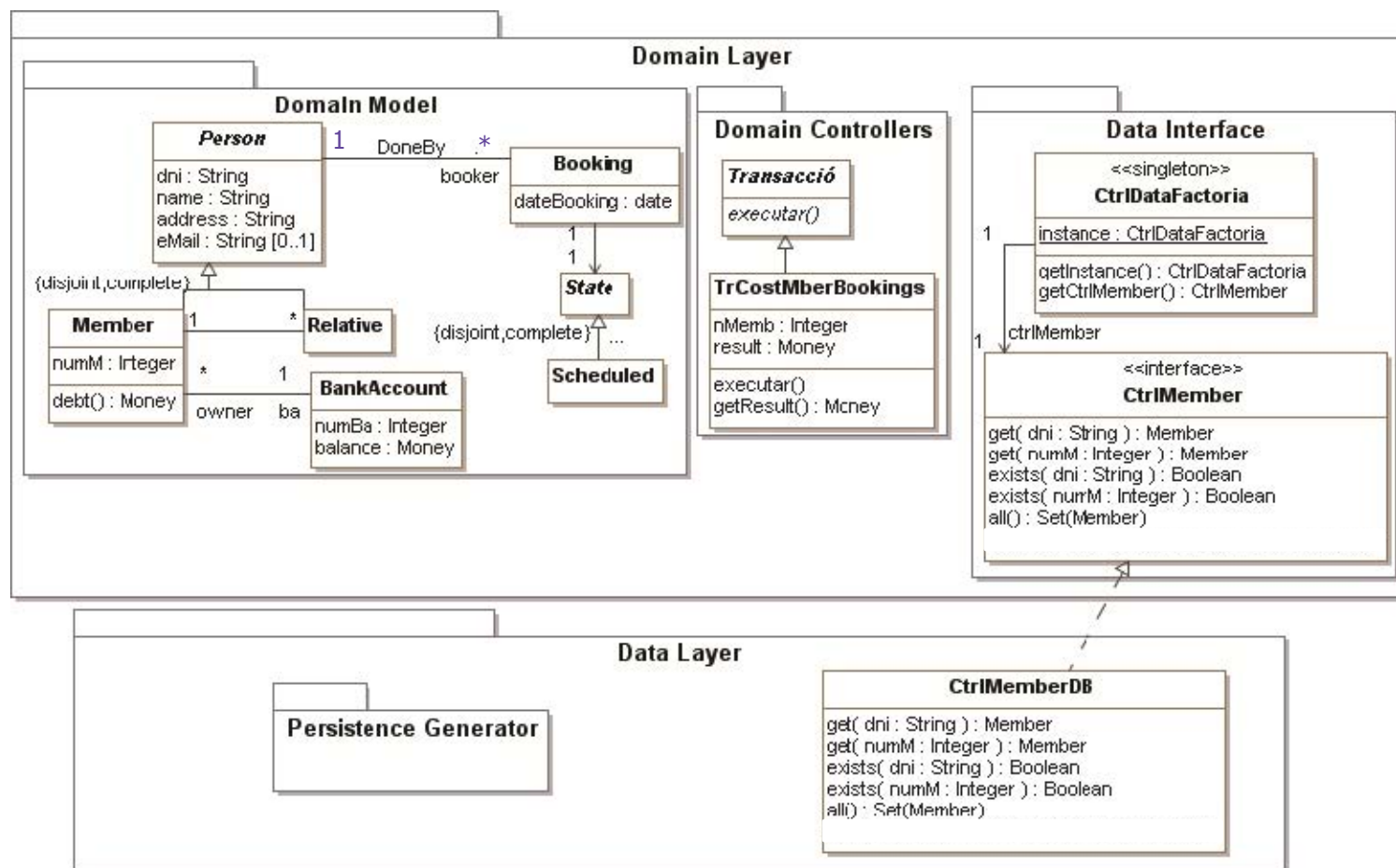
- The designer defines the correspondence among the normalized class diagram and the relational schema
- The Data Mapper maintains the correspondence between the objects and the tables in the database.
- The Data Mapper knows (almost) automatically the updates made at the objects and knows how to propagate them into the tables.
- Different products implement the Data Mapper pattern in different ways:
 - Hibernate, Spring, EclipseLink, ...

Automatic generation of persistence:

Data Mapper

- There is not need of an specific design of the data layer.
- Only, the implementetation of Data Interfaces of Domain Layer need to be added at the Data Layer.
- The sequence diagrams in the design of the Domain Layer do not change
- It is possible to navigate object relationships transparently.
Related objects are automatically loaded as needed.
- Object loads are configurable.
- Concurrency support.
- Cache managment.
- Transaction management and Isolation.

Automatic generation of persistence: Domain Layer and Data Layer, example



Automatic generation of persistence:

Domain Layer and Data Layer, example

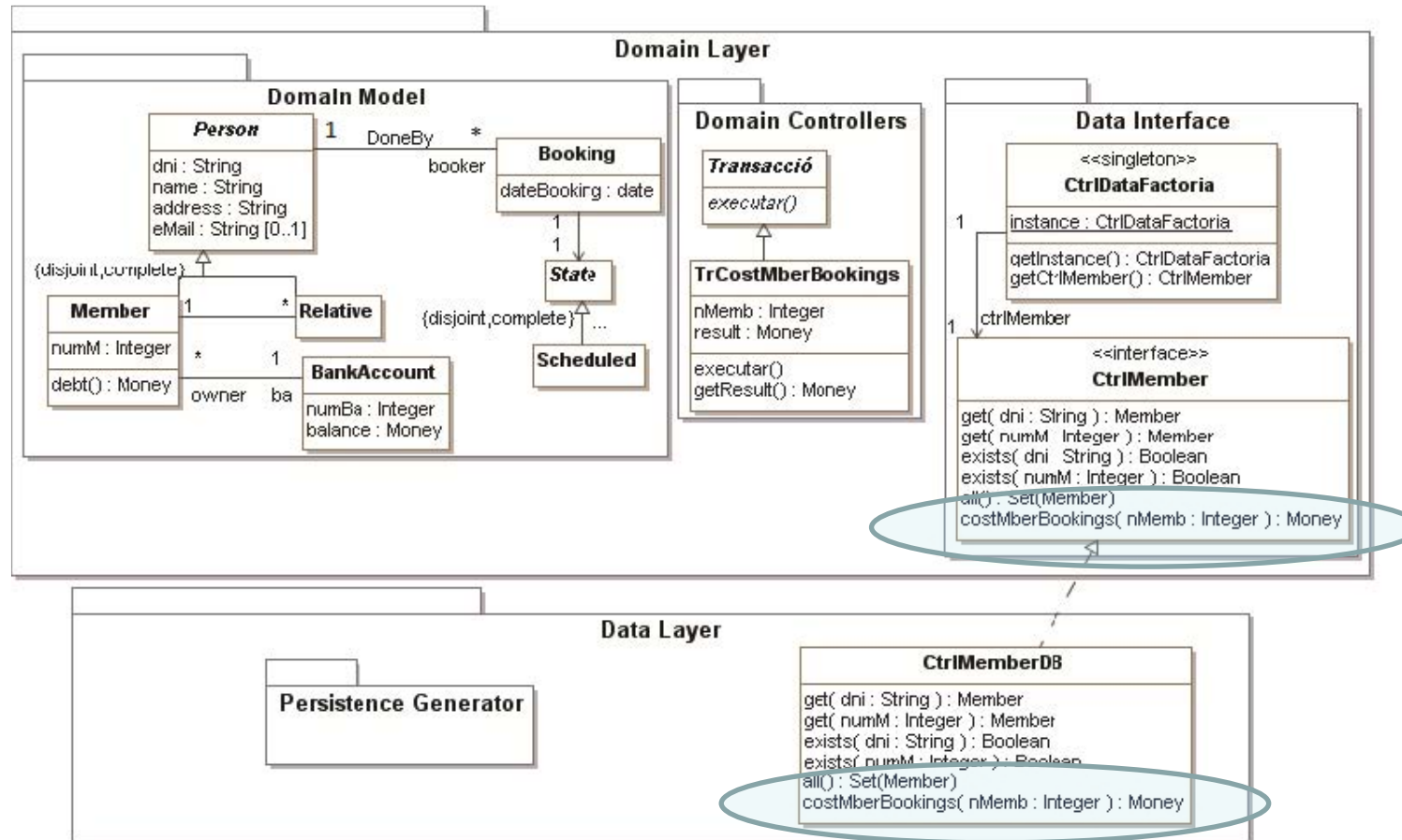
- Each Data Interface controller of a Domain Class is implemented in the Data Layer.
 - Operations: get, exists, all
- Sequence diagrams designed for the Domain Layer do not change.
- Data Mapper is in charge of:
 - load objects from tables of the database (when: get, all, automatically in navigation)
 - maintaining the correspondence between the objects and the tables.
 - transaction and concurrence management, cache support, etc.

Automatic generation of persistence:

Variant: Arbitrary query operations on the Data layer

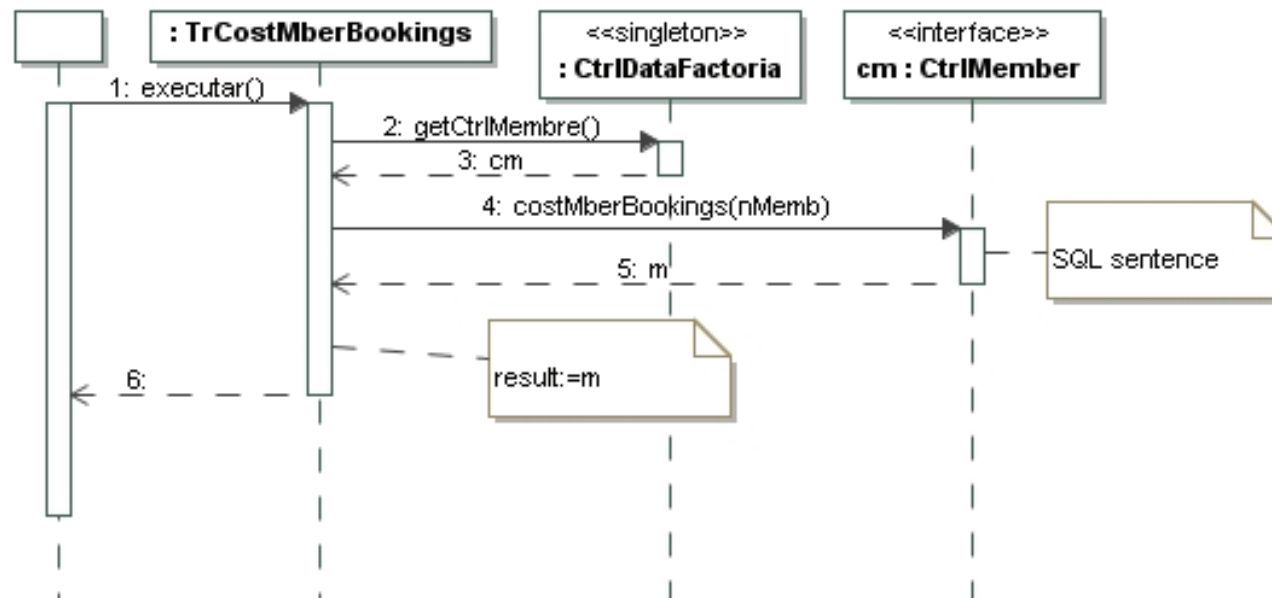
- The automatic generation of persistence:
 - Very comfortable for updating the database but not so much for certain queries
 - Sequence diagrams in some queries cumbersome and error-prone
 - In some queries potential loss of efficiency
- Data Layer offers query operations required by a problem at hand
 - Arbitrary query operations => operations specific for resolving a query
 - Operations: get, exists, all, other operations that can return concrete data. c
- Current generators of persistence offer this possibility:
 - Ex: Hibernate, *createQuery*

Automatic generation of persistence: Arbitrary query operations on the Data layer, example



Automatic generation of persistence:

Arbitrary query operations on the Data layer, example



Direct Design of of persistence

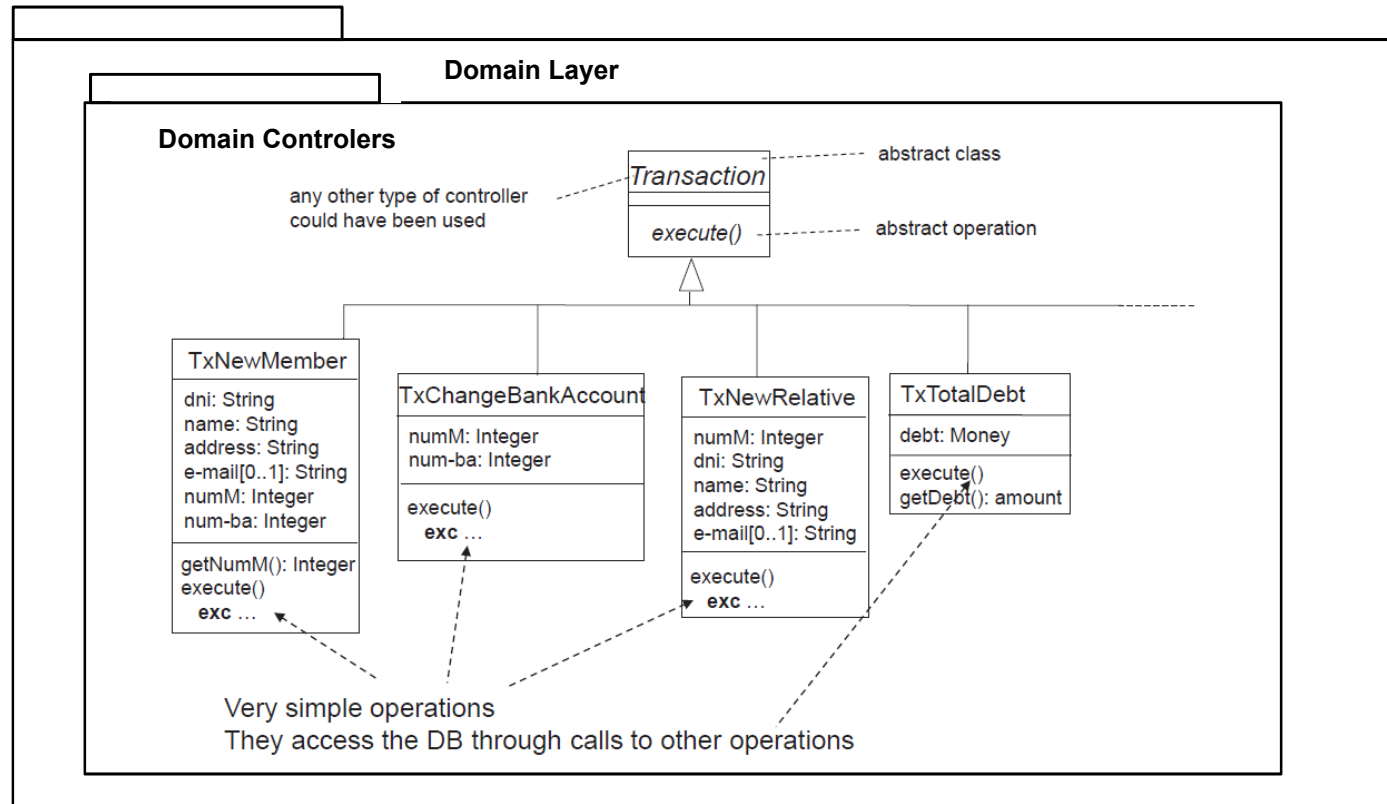
Strategies for managing persistence:

Direct design of persistence

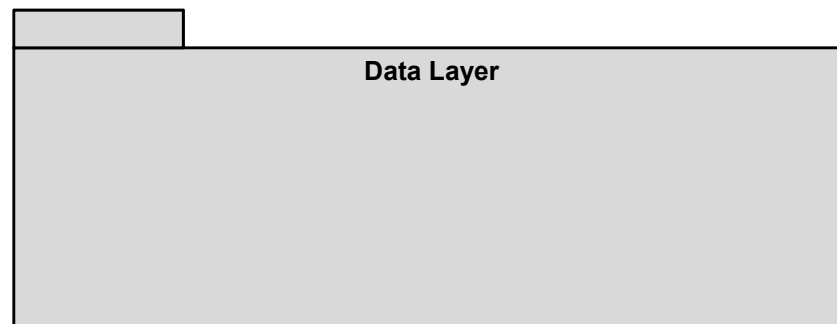
- Direct design:
 - Software design is elaborated taking into account the DBMS that will be used in the implementation.
 - Therefore, the design will look different depending on the model of the DBMS used (relational model, etc.)
- Characteristics:
 - The designer needs to know the DBMS used, in order to design the Domain and Data Layers
 - The full functionalities of the DBMS may be exploited (tables, stored procedures, triggers, etc.)
 - The resulting design is similar for all the DBMS that correspond to the same model.

It is used jointly with the Transaction Script design pattern

Direct design of persistence: Transaction Script Pattern



Different combinable
strategies and patterns



Person (dni, name, address, e-mail)
Relative (dni, member)
Member (dni, numM)
...

Direct design of persistence:

Domain Controller Strategies

- Domain controller as pure intermediary
 - The controller delegates on a stored procedure that takes on all the responsibility
- Domain controller with plain logic
 - The controller calls operations that access the DB for obtaining and updating data, by using data access patterns:
 - Table Data Gateway pattern
 - Row Data Gateway pattern
 - Active Record pattern
 - The controller performs simple manipulations over these data.

It is possible to combine both strategies

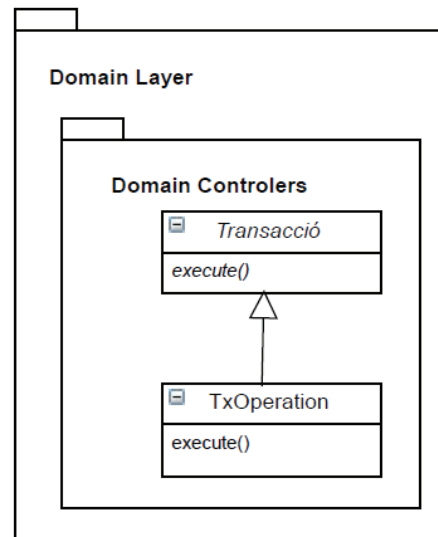
Direct design of persistence:

Domain controller as pure intermediary

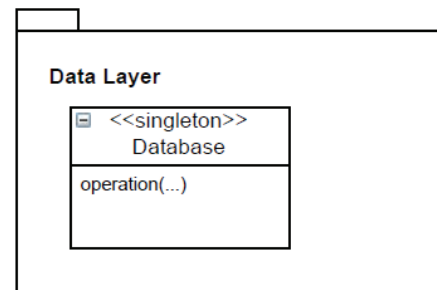
- Domain controller as pure intermediary
 - The controller delegates on a stored procedure that takes on all the responsibility

Direct design of persistence:

Domain controller as pure intermediary



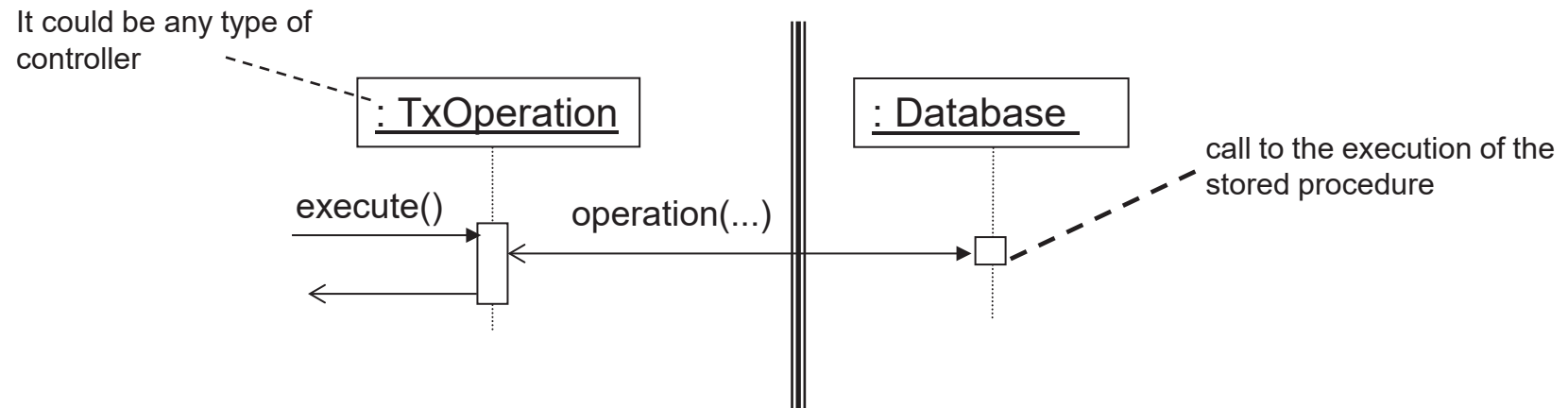
any other type of controller could have been used



tables,
stored procedures: operation
...

Direct design of persistence:

Domain controller as pure intermediary



Direct design of persistence:

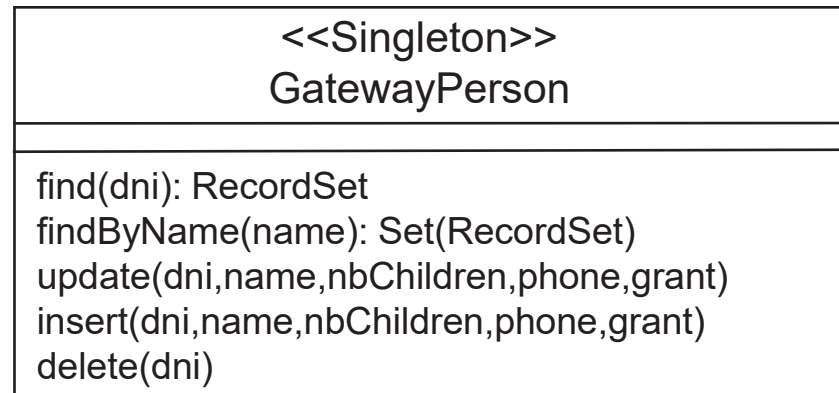
Domain controller with plain logic - Data access patterns

- Table Data Gateway
 - A singleton class for each data base table.
 - Encapsulates the access to a table.
- Row Data Gateway
 - A class for each data base table.
 - An instance of the class corresponds to a table row
 - Encapsulates the update of a row.
 - The find operations may be class operations or be defined in a singleton class Finder.
- Active Record
 - Row data gateway with operations that come from the problem domain.

Direct design of persistence:

Domain controller with plain logic - Table Data Gateway

One singleton class per table, which encapsulates the access to a table
The object instance allows to handle all the rows in the table



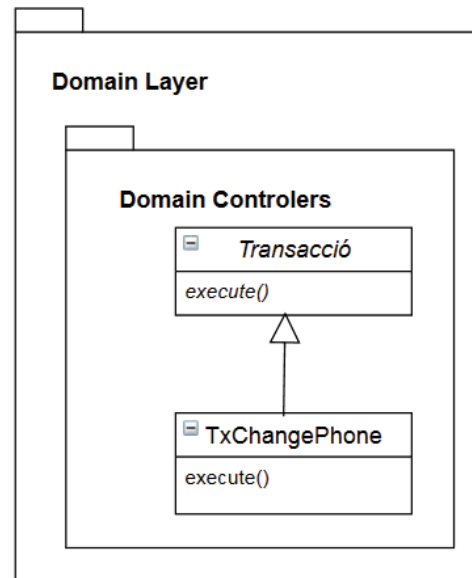
RecordSet returned by the find
operations contain the
data of a Person

Diferent alternatives of how to return data:

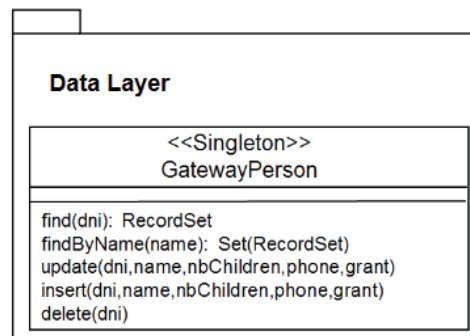
- RecordSet
- Data Transfer Object.

Direct design of persistence:

Domain controller with plain logic - Table Data Gateway

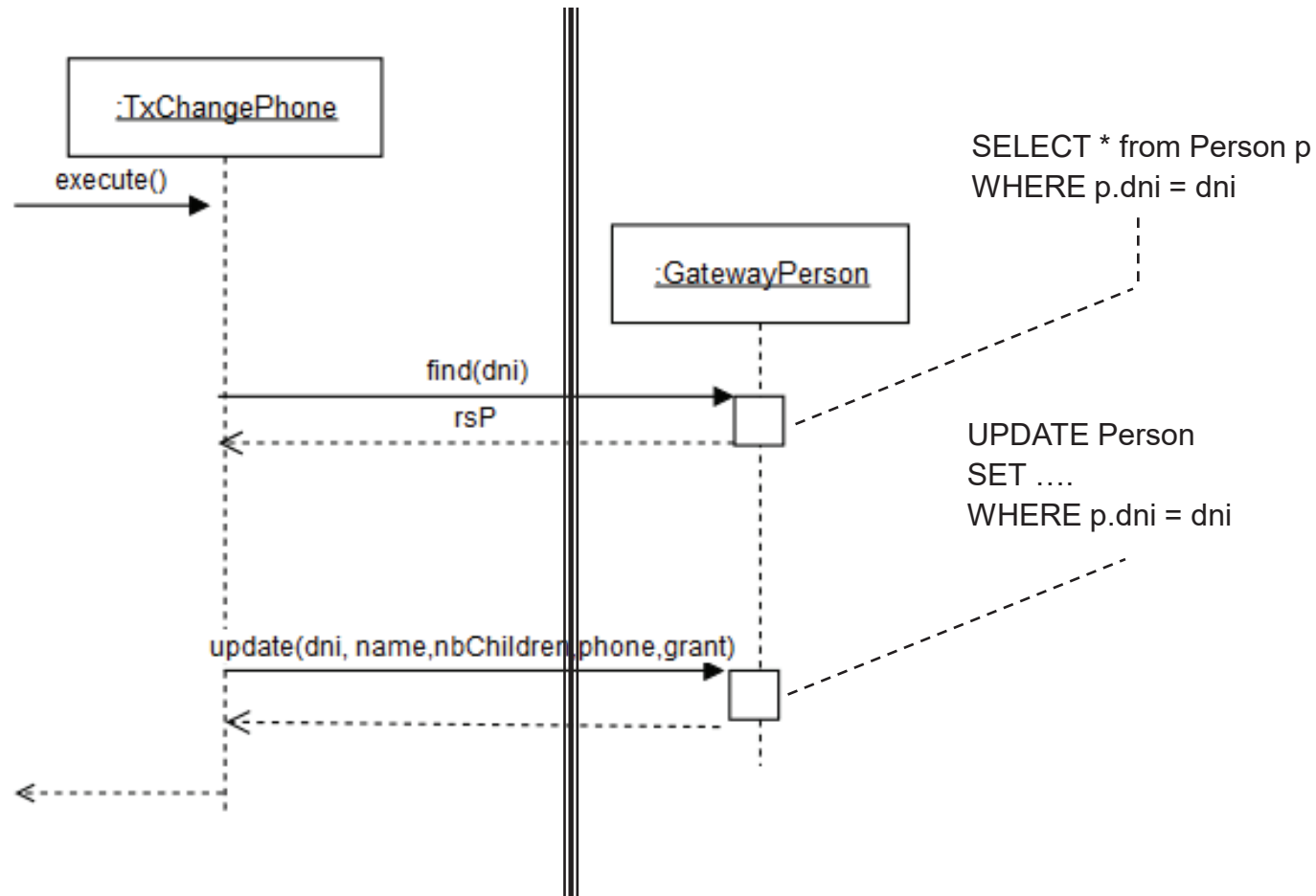


any other type of controller could have been used



Direct design of persistence:

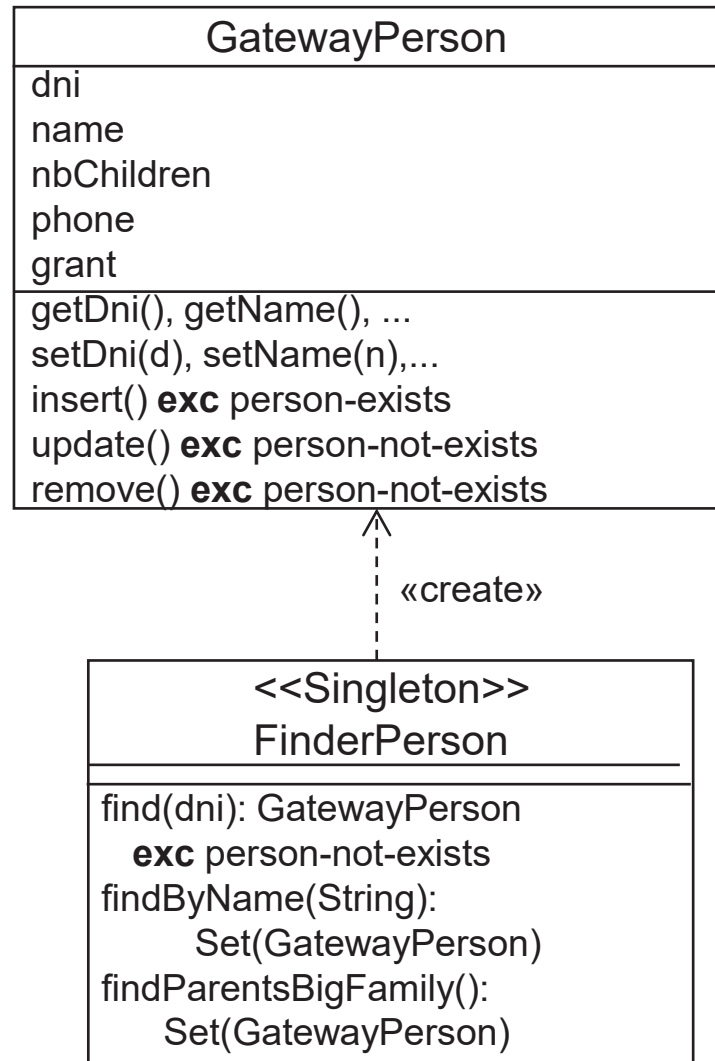
Domain controller with plain logic - Table Data Gateway



Direct design of persistence:

Domain controller with plain logic - Row Data Gateway

- A gateway class for each table
- Object instances correspond to table rows.
- Each attribute of the class correspond to a column in the table
- A singleton Finder class for each table

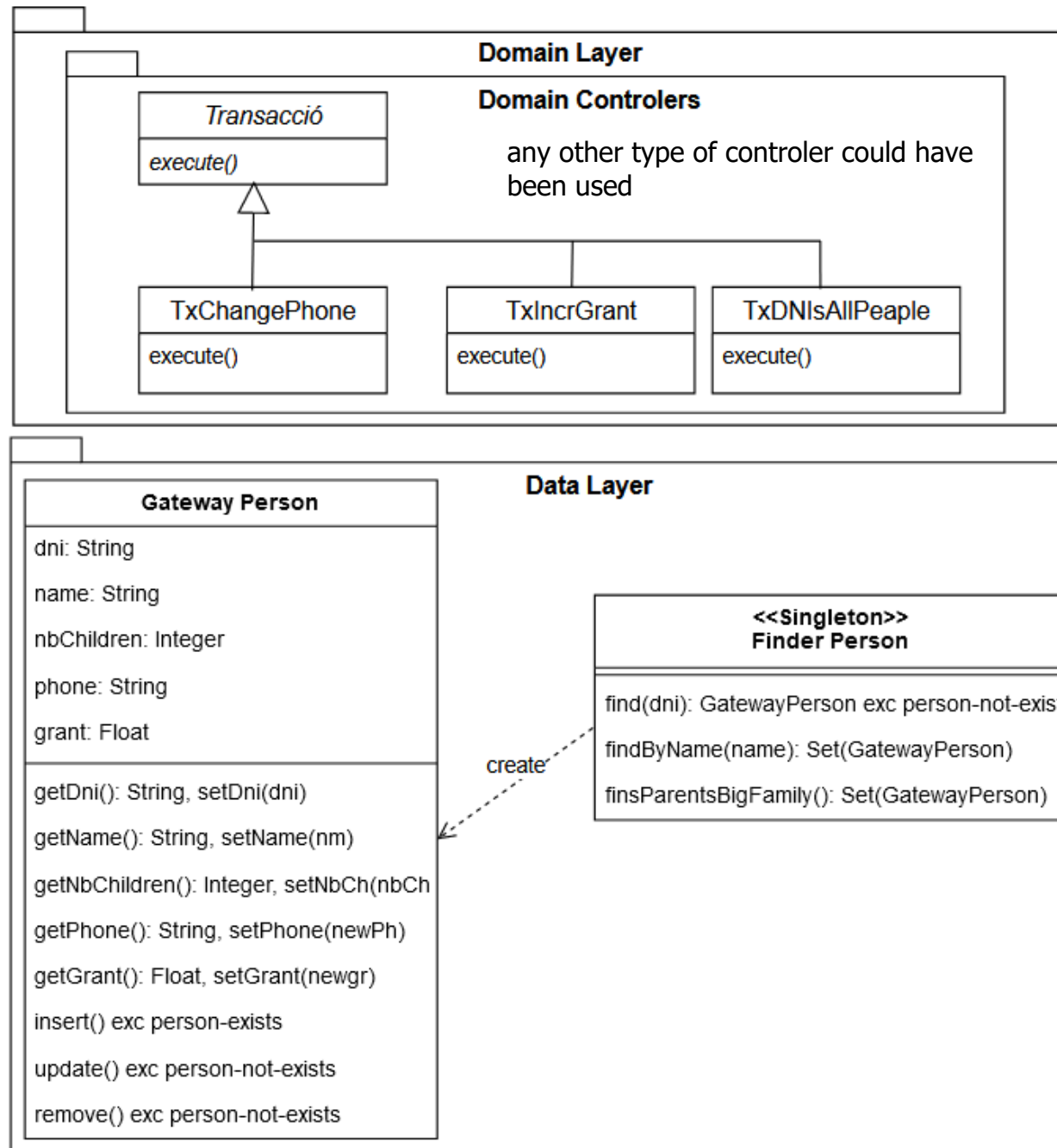


insert, update or
remove SQL sentences
of table Person

select SQL sentences
of table Person

Direct design of persistence:

Domain controler with plain logic – Row Data Gateway



Domain controllers:

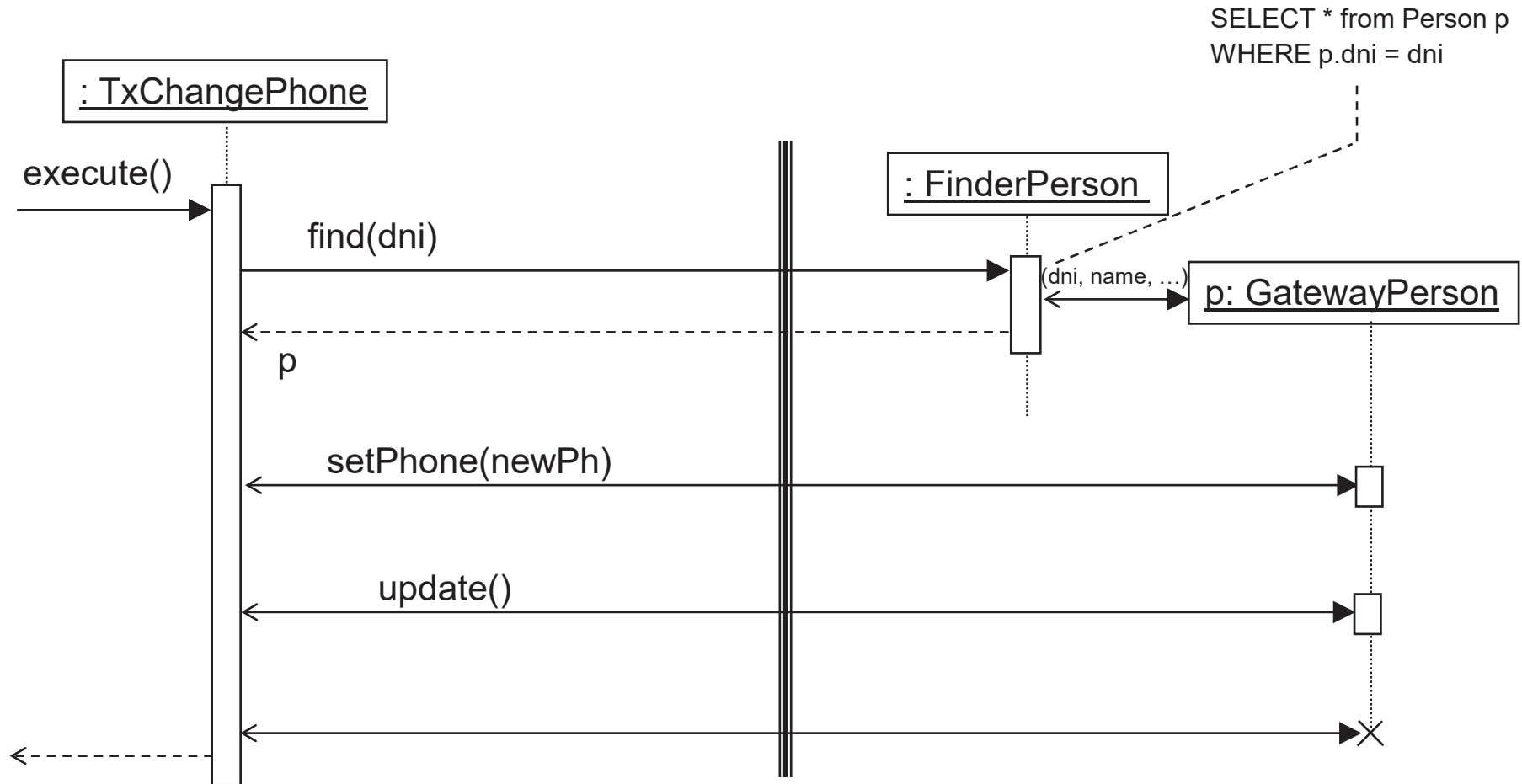
- Create gateways for inserting objects
- Ask finders for obtaining objects (gateways)
- Manipulate these objects in main memory
- Pull the objects back to the Data layer using the gateway

Finder operations:

- Restricted to the table scope
- Just gateways allowed as result

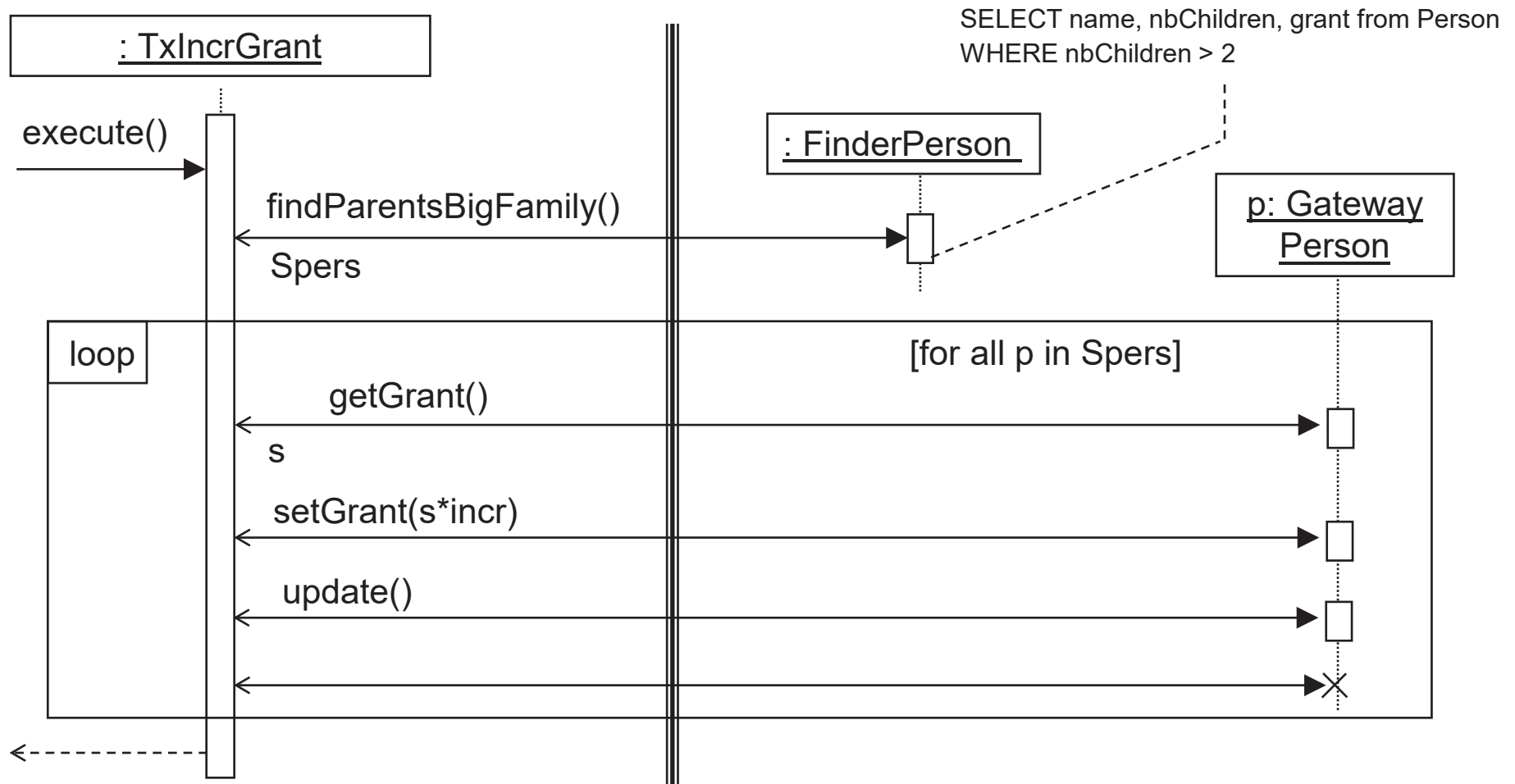
Direct design of persistence:

Domain controller with plain logic - Row Data Gateway



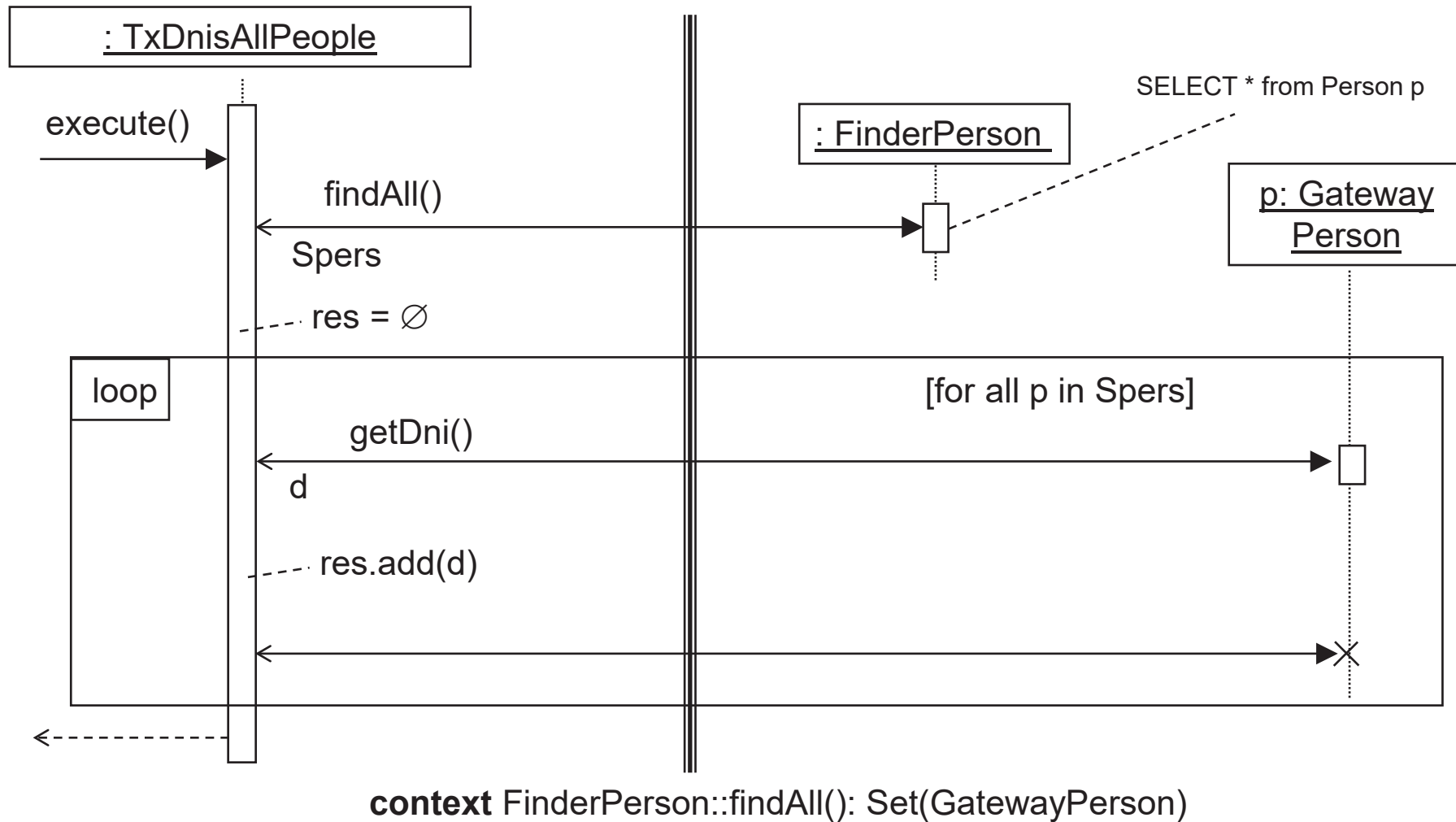
Direct design of persistence:

Domain controller with plain logic - Row Data Gateway



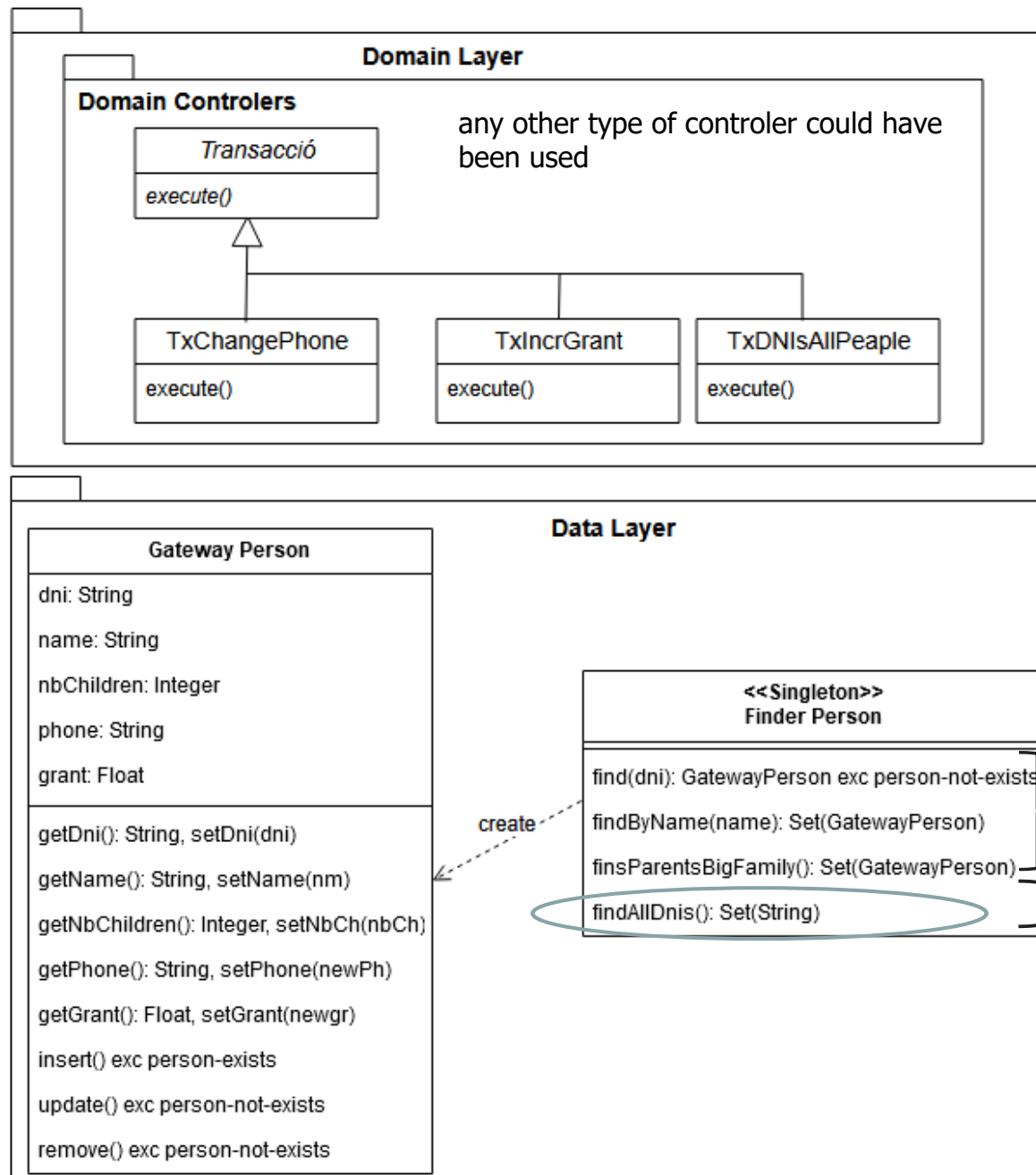
Direct design of persistence:

Domain controler with plain logic - Row Data Gateway



Direct design of persistence:

Domain controller with plain logic – Row Data Gateway, Variant: arbitrary query operations



Simple query operations:

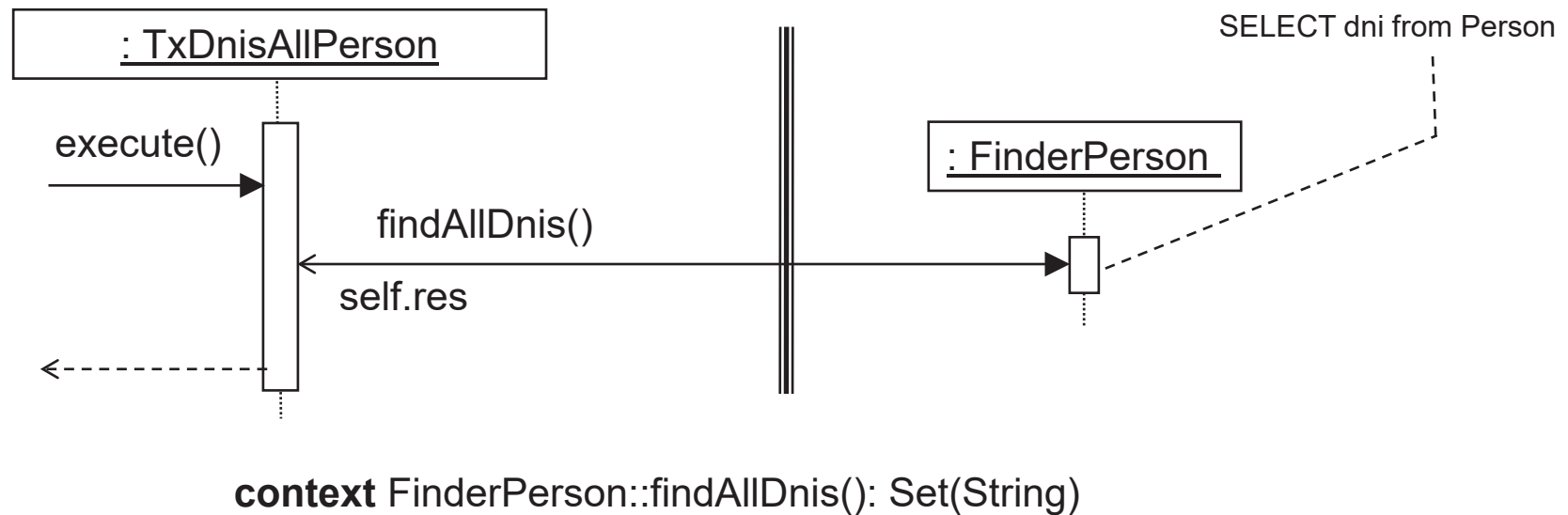
- Restricted to the table scope
- Just gateways allowed as result

Arbitrary query operations:

- Computation of results
- NOT restricted to the table scope
- Types different of Gateways are allowed in the results

Direct design of persistence:

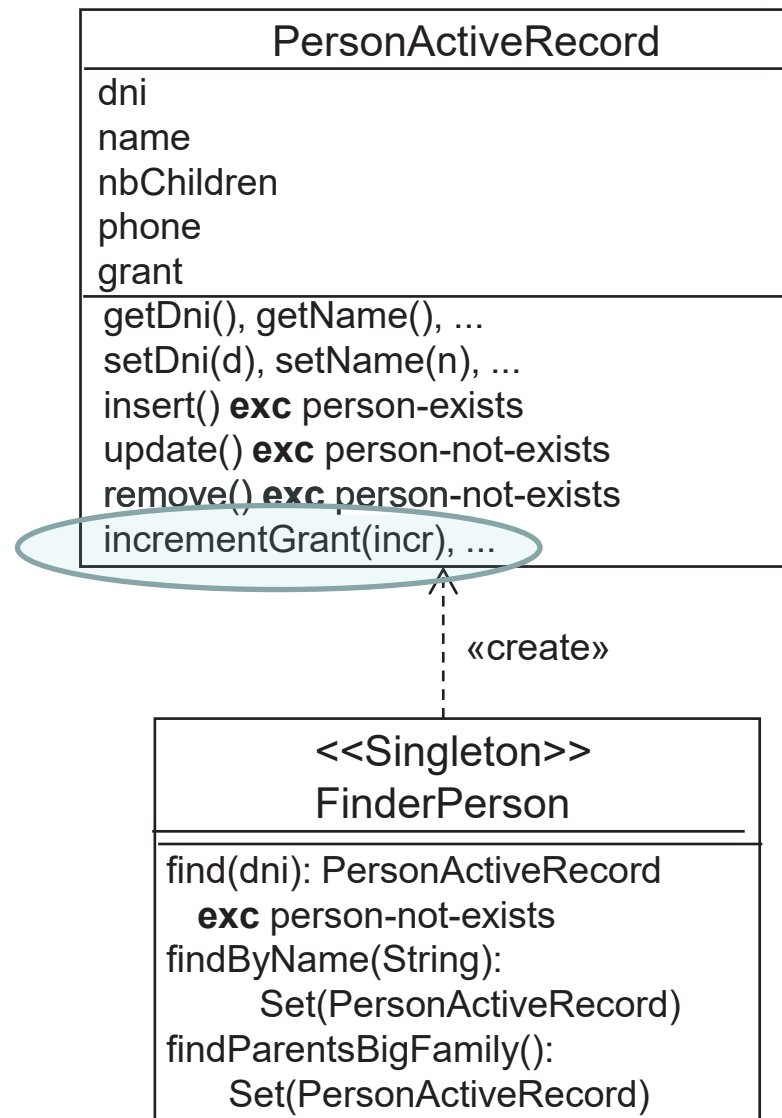
Domain controller with plain logic – Row Data Gateway, Variant: arbitrary query operations



Direct design of persistence:

Domain controller with plain logic – Active Record

- Similar to Row Data Gateway
- An active record class for each table.
- Object instances correspond to table rows.
- Each attribute of the class corresponds to a table column
- Extra operations in active record classes: Implement some pieces of business logic
- A finder singleton class for each table.

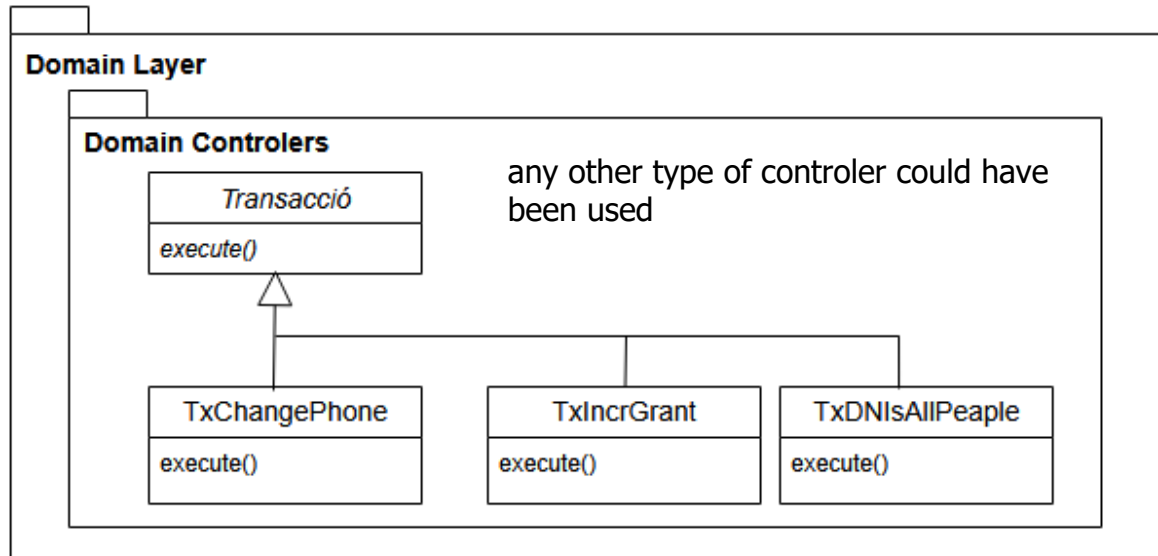


insert, update or
remove SQL sentences
of table Person

select SQL sentences
of table Person

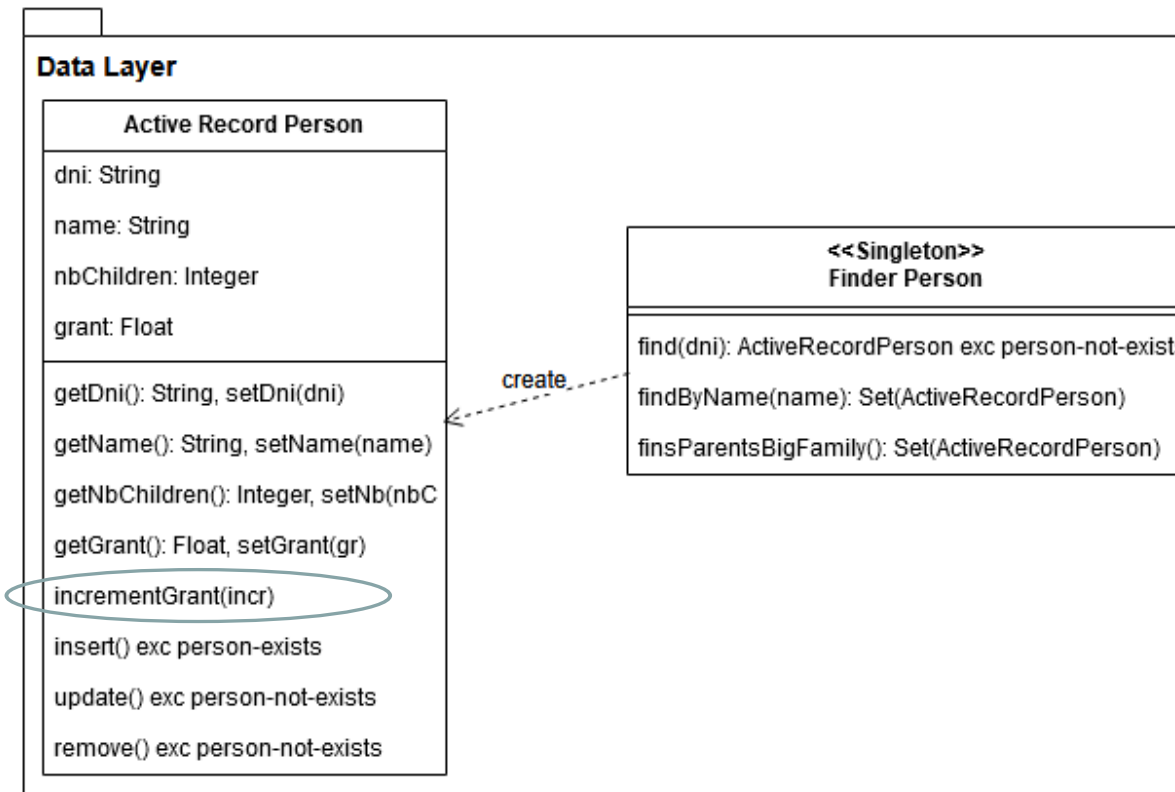
Direct design of persistence:

Domain controller with plain logic – Active Record



Domain controllers:

- Create gateways for inserting objects
- Ask finders for obtaining objects (gateways)
- Manipulate these objects in main memory
- Pull the objects back to the Data layer using the gateway

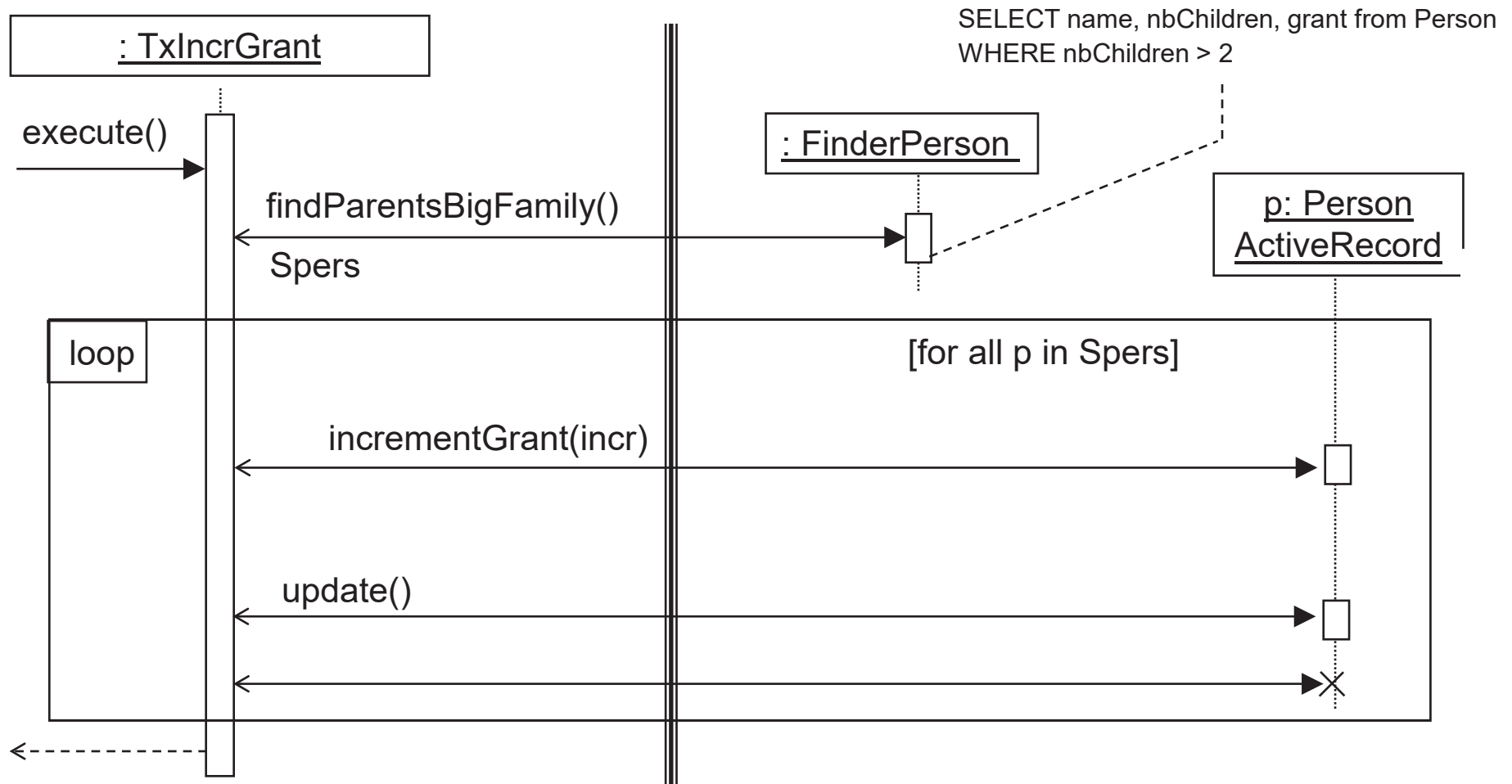


Finder operations:

- Restricted to the table scope
- Just gateways allowed as result

Direct design of persistence:

Domain controller with plain logic – Active Record



Direct design of persistence:

Summary of Strategies and Patterns

- Domain controller as pure intermediary (Stored Procedures)
- Domain controller with plain logic. Patterns:
 - Table Data Gateway
 - 1 singleton class per table
 - Row Data Gateway - “passarel.la fila”
 - 1 Gateway class per table, 1 instance per row
 - Gateway Operations: getAtr, setAtr, insert, update, remove
 - 1 singleton Finder class per table
 - Variant: with arbitrary query operations in the Finder classes.
 - Active Record
 - 1 Active Record class per table, 1 instance per row
 - Operations: getAtr, setAtr, insert, update, remove, other operations specific of the domain.
 - 1 singleton Finder class per table
 - Variant: with arbitrary query operations in the Finder classes.

References

- M. Fowler
Patterns of Enterprise Application Architecture
Addison-Wesley, 2003
- H. Garcia-Molina, J. Ullman, J. Widom
Database Systems Implementation
Prentice-Hall, 2000.
- J. Melton, A. Eisenberg
Understanding SQL and Java Together
Morgan-Kaufmann, 2000.
- Christian Bauer, Gavin King
Hibernate in Action
Manning Publications Company, 2004